

# Qualidade e Segurança de Software

CST em Análise e Desenvolvimento de Sistemas

Professor: Arliones Hoeller Jr.

[arliones.hoeller@ifsc.edu.br](mailto:arliones.hoeller@ifsc.edu.br)

# Licenciamento



Slides licenciados sob [Creative Commons "Atribuição 4.0 Internacional"](https://creativecommons.org/licenses/by/4.0/)

# Conceitos de qualidade

# O que é Qualidade?

- O *American Heritage Dictionary* define qualidade como “uma característica ou atributo de algo”.
- Três tipos de qualidade no software:
  - **Qualidade de projeto:** Abrange requisitos, especificações e projeto do sistema.
  - **Qualidade de conformidade:** Foca na implementação.
  - **Satisfação do usuário** = produto em conformidade + boa qualidade + entrega no prazo e dentro do orçamento.

# Qualidade – Visão Filosófica

- Robert Pirsig (PIRSIG, 1974) comentou algo como (adaptado para o slide):  
*“Qualidade... você sabe o que é, mas não sabe o que é. Algumas coisas têm mais qualidade do que outras. Mas, se você tentar definir qualidade, tudo desaparece.”*
- Se ninguém sabe o que é qualidade, como sabemos que ela existe?

# Qualidade – Visões Pragmáticas

- **Transcendental:** Qualidade é algo que você reconhece, mas não consegue definir explicitamente.
- **Usuário:** Qualidade como a capacidade de atender às metas específicas do usuário final.
- **Fabricante:** Qualidade como a aderência às especificações originais.
- **Produto:** Qualidade vinculada às características inerentes de um produto (e.g., funções e recursos).
- **Baseada em valor:** Medida de quanto um cliente está disposto a pagar por um produto.

# Qualidade de Software

- Definição: Um processo de software eficaz que cria um produto útil com valor mensurável para produtores e usuários.
- Vantagens:
  - Maior receita com produtos de software.
  - Melhor lucratividade ao suportar processos de negócios.
  - Melhor disponibilidade de informações críticas.

# Qualidade em Uso – ISO25010:2017

- **Eficácia:** Precisão e completude com que usuários atingem seus objetivos.
- **Eficiência:** Recursos gastos para alcançar objetivos.
- **Satisfação:** Utilidade, confiança, prazer, conforto.
- **Liberdade de riscos:** Mitigação de riscos econômicos, de saúde e ambientais.
- **Cobertura de contexto:** Completude, flexibilidade.



# Qualidade do Produto – ISO25010:2017

- **Adequação funcional:** Completo, correto, apropriado.
- **Eficiência de desempenho:** Tempo, uso de recursos, capacidade.
- **Compatibilidade:** Coexistência, interoperabilidade.
- **Usabilidade:** Aprendizado, operação, estética, acessibilidade.
- **Confiabilidade:** Maturidade, disponibilidade, tolerância a falhas.
- **Segurança:** Confidencialidade, integridade, autenticidade.
- **Manutenibilidade:** Reutilização, modificabilidade, testabilidade.
- **Portabilidade:** Adaptabilidade, instalabilidade, substituíbilidade.

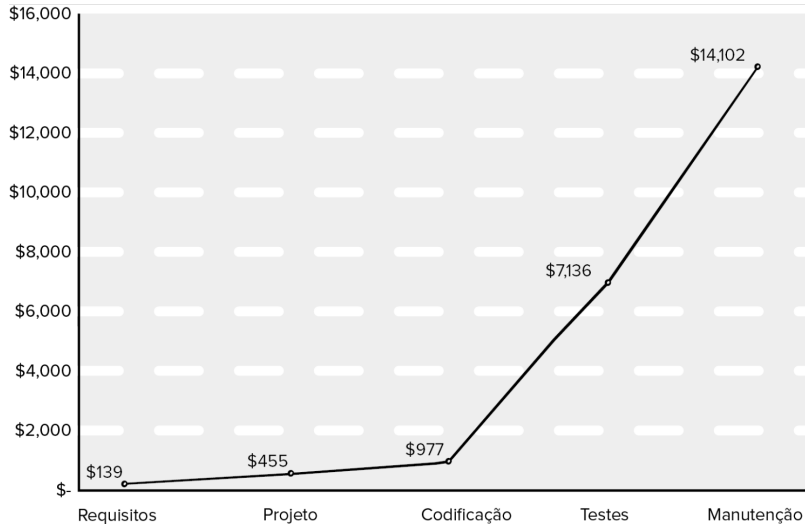
# O Dilema da Qualidade de Software

- Software de baixa qualidade pode ser rejeitado pelo mercado.
- Software excessivamente perfeito pode ser inviável devido ao custo e tempo.
- O desafio é encontrar o equilíbrio entre qualidade suficiente e custos controlados.

# Custos de Qualidade

- **Prevenção:** Planejamento, revisões técnicas formais, treinamento.
- **Avaliação:** Revisões, coleta de dados, testes.
- **Falhas internas:** Retrabalho, reparo, análise de falhas.
- **Falhas externas:** Suporte ao cliente, substituição, garantias.

# Custos Relativos para Encontrar e Corrigir Defeitos



Fonte: (PRESSMAN; MAXIM, 2021)

# Negligência e Responsabilidade

- Uma entidade governamental ou corporativa contrata uma grande empresa de desenvolvimento ou consultoria de software para analisar requisitos, projetar e construir um sistema baseado em software.
- O sistema pode suportar uma função corporativa principal (ex.: gestão de pensões) ou governamental (ex.: administração de saúde ou segurança nacional).
- O trabalho começa com as melhores intenções de ambas as partes, mas quando o sistema é entregue:
  - Está atrasado.
  - Não oferece as funcionalidades e características desejadas.
  - É propenso a erros.
  - Não é aceito pelo cliente.
- O resultado? Litígios seguem.

# Qualidade, Risco e Segurança

- Software de baixa qualidade aumenta os riscos tanto para desenvolvedores quanto para usuários finais.
- Sistemas entregues com atraso, sem funcionalidade completa e que não atendem às expectativas dos clientes frequentemente levam a litígios.
- Software de baixa qualidade é mais vulnerável a ataques e aumenta os riscos de segurança para a aplicação após sua implantação.
- Sistemas seguros não podem ser construídos sem um foco claro na qualidade (segurança, confiabilidade e dependabilidade) durante o design.
- Software de baixa qualidade pode conter tanto falhas arquiteturais quanto problemas de implementação (bugs).

# Impacto das Decisões de Gerência

- **Estimativas:** Prazos irreais levam a atalhos que reduzem a qualidade.
- **Cronogramas:** Ignorar dependências pode comprometer o projeto.
- **Riscos:** Responder a crises sem planejamento gera caos e reduz a qualidade.

# Alcançando Qualidade de Software

- Qualidade resulta de boa gestão de projetos e práticas sólidas de engenharia.
- Planejamento do projeto deve incluir:
  - Técnicas explícitas para gestão de qualidade e mudanças.
  - Controle de qualidade com inspeções, revisões e testes.
  - Garantia de qualidade com auditorias e relatórios para decisões proativas.



## Questões para discutir

- 1 Usando a definição de qualidade de software proposta, você acredita que seja possível criar um produto útil que gere valor mensurável sem usar um processo eficaz? Justifique sua resposta.
- 2 Descreva o dilema da qualidade de software com suas próprias palavras.
- 3 O que é software “bom o suficiente”? Cite uma empresa específica e produtos específicos que você acredita terem sido desenvolvidos usando essa filosofia.
- 4 Considerando cada um dos quatro aspectos do custo da qualidade, qual você acredita ser o mais caro? Por quê?
- 5 Faça uma busca na Internet e encontre três outros exemplos de “riscos” para o público que podem ser diretamente atribuídos à baixa qualidade de software. Pense em iniciar sua pesquisa em <http://catless.ncl.ac.uk/risks>.

# **Teste de software – Nível de componentes**

# Abordagem Estratégica para Testes

- Realize revisões técnicas eficazes para eliminar muitos erros antes de iniciar os testes.
- Os testes começam no nível de componente e avançam “para fora”, em direção à integração do sistema completo.
- Técnicas de teste diferentes são apropriadas para diferentes abordagens de engenharia de software.
- Testes são conduzidos pelo desenvolvedor e, para grandes projetos, por um grupo de testes independente.
- Testar e depurar são atividades diferentes, mas a depuração deve ser incluída em qualquer estratégia de teste.

# Verificação e Validação

- **Verificação:** Garantir que o software implementa corretamente uma função específica:

*“Estamos construindo o produto corretamente?”*

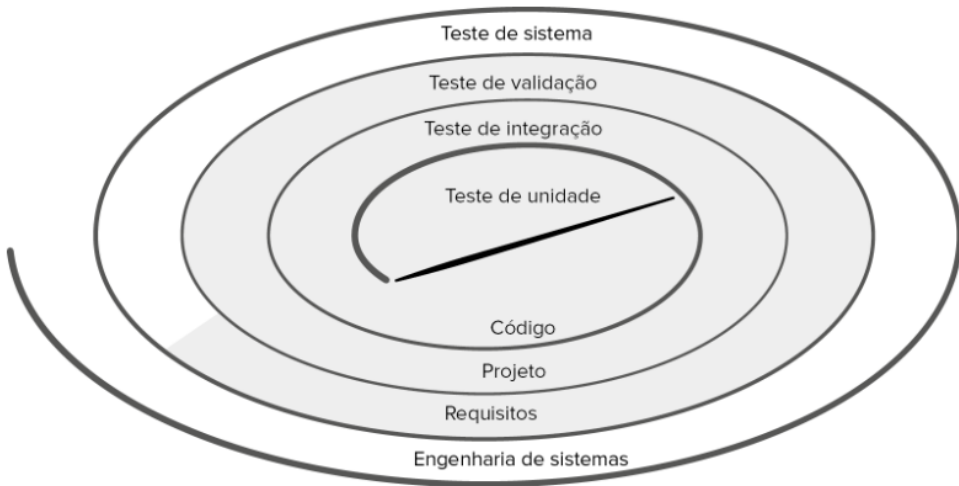
- **Validação:** Garantir que o software construído seja rastreável aos requisitos do cliente.

*“Estamos construindo o produto certo?”*

# Organização para Testes

- Desenvolvedores de software são responsáveis por testar componentes individuais.
- Um grupo de testes independente (GTI) se envolve após a conclusão da arquitetura de software.
- O GTI remove problemas associados ao fato de o construtor testar o que construiu.
- O GTI é pago para encontrar erros, colaborando com os desenvolvedores para garantir testes completos.

# Estratégia de Testes



Fonte: (PRESSMAN; MAXIM, 2021)

# Testando: a Visão Geral

- **Teste de Unidade:** Começa no centro da espiral, concentrando-se em cada unidade (por exemplo, componente, classe ou objeto de conteúdo) conforme são implementadas no código-fonte.
- **Teste de Integração:** O foco está no design e na construção da arquitetura de software, avançando para a próxima etapa da espiral.
- **Teste de Validação:** Verifica se os requisitos estabelecidos na modelagem de requisitos são atendidos pelo software construído.
- **Teste de Sistema:** O software e outros elementos do sistema são testados como um todo para garantir sua funcionalidade e desempenho.

# Prática: Teste de Unidade

- Teste de unidade é a primeira atividade de teste a ser realizada.
- Objetivo: verificar se cada unidade de software (componente, classe ou objeto) está funcionando corretamente.
- Testes de unidade são conduzidos pelo desenvolvedor.
- Testes de unidade são normalmente automatizados.
- Nós utilizaremos o JUnit para testes de unidade em Java.



# Prática: JUnit

- JUnit é um framework de teste de unidade para a linguagem de programação Java.
- Foi criado por Erich Gamma e Kent Beck.
- É uma implementação da arquitetura xUnit para testes de unidade.
- É distribuído sob a licença de software livre Common Public License Version 1.0.

## Prática: JUnit no IntelliJ

- No IntelliJ, crie um novo projeto Java e, neste projeto, crie uma classe a ser testada, como a classe Calculator no exemplo.

```
public class Calculator {  
  
    static double add(double... operands) {  
        return DoubleStream.of(operands)  
            .sum();  
    }  
  
    static double multiply(double... operands) {  
        return DoubleStream.of(operands)  
            .reduce(1, (a, b) -> a * b);  
    }  
}
```

## Prática: JUnit no IntelliJ

- Configure o JUnit no projeto IntelliJ adicionando a dependência no arquivo `build.gradle`:

```
testImplementation 'org.junit.jupiter:junit-jupiter'
```

- Atualize a configuração *gradle* do projeto.

## Prática: JUnit no IntelliJ

- Crie a classe de testes.
- Clique com o botão direito sobre o nome da classe a ser testada e selecione Show Context Actions > Create Test.
- Selecione a opção JUnit5, selecione os métodos a serem testados e clique em OK.
- Isso criará a classe de testes com os métodos de teste vazios para que os testes sejam implementados.

```
class CalculatorTest {  
    @Test  
    void add() { }  
    @Test  
    void multiply() { }  
}
```

## Prática: JUnit no IntelliJ

- Agora basta implementar os testes para a classe `Calculator`.
- Para testar a classe, o framework de testes usa *Afirmções (Assertions)*.
- Estas afirmações consistem em afirmar que um valor esperado é igual ao valor retornado pelo método a ser testado.
- Por exemplo, para o método `soma`, o teste pode ser implementado da seguinte forma:

```
@Test
void add() {
    assertEquals(5, Calculator.add(2, 3));
}
```

- O método `assertEquals` compara o valor esperado (5) com o valor retornado pelo método `add(2, 3)`, ou seja, verifica se  $2 + 3 = 5$ .

## Prática: JUnit no IntelliJ

- A API do JUnit disponibiliza diferentes métodos da “família” assert que permitem testar diferentes condições:

| Método                         | Descrição                                               |
|--------------------------------|---------------------------------------------------------|
| <code>assertArrayEquals</code> | Verifica se dois arrays são iguais.                     |
| <code>assertEquals</code>      | Verifica se dois valores são iguais.                    |
| <code>assertTrue</code>        | Verifica se uma condição é verdadeira.                  |
| <code>assertFalse</code>       | Verifica se uma condição é falsa.                       |
| <code>assertNull</code>        | Verifica se um valor é nulo.                            |
| <code>assertNotNull</code>     | Verifica se um valor não é nulo.                        |
| <code>assertSame</code>        | Verifica se duas variáveis apontam para o mesmo objeto. |
| <code>assertNotSame</code>     | Verifica se duas variáveis apontam objetos diferentes.  |

# Prática: JUnit no IntelliJ

- Outros métodos interessantes são o `assertAll` e o `assertThrows`:
- O `assertAll` permite agrupar várias afirmações em um único bloco.
- O `assertThrows` verifica se uma exceção é lançada por um método.

```
@Test
void add() {
    assertAll("add",
        () -> assertEquals(5, Calculator.add(2, 3)),
        () -> assertEquals(0, Calculator.add()),
        () -> assertEquals(7, Calculator.add(2, -3, 5))
    );
    assertThrows(IllegalArgumentException.class, () -> Calculator.add(null));
    assertThrows(IllegalArgumentException.class, () -> Calculator.add());
    assertThrows(IllegalArgumentException.class, () -> Calculator.add(1));
}
```

## Exercício: Testes da classe Pessoa

- Crie uma nova classe chamada Pessoa com os atributos `nome` e `cpf` e os métodos abaixo:

| Método                               | Regra de Negócio                                                                                                                                                                                                                                |
|--------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| construtor<br><code>setNome()</code> | Toda Pessoa deve ter <code>nome</code> e <code>cpf</code> ; deve utilizar os <i>setters</i> . Seta o atributo <code>nome</code> ; o nome não pode ser nulo ou vazio; o nome deve conter ao menos duas palavras; lançar exceção em caso de erro. |
| <code>setCpf()</code>                | Seta o atributo <code>cpf</code> ; o CPF não pode ser nulo ou vazio; o CPF deve ser válido; lançar exceção em caso de erro.                                                                                                                     |
| <code>getNome()</code>               | Retorna o nome da pessoa.                                                                                                                                                                                                                       |
| <code>getCpf()</code>                | Retorna o CPF da pessoa.                                                                                                                                                                                                                        |
| <code>validarCpf()</code>            | Verifica se um CPF é válido e retorna valor booleano (implementação fornecida).                                                                                                                                                                 |



## Exercício: Robô de exploração I

- Modele uma classe `RoboExplorador` com as características abaixo.
- O Robô tem: um nome, sua coordenada atual em um plano cartesiano, um ângulo identificando para onde está apontando sua frente e um medidor de bateria que indica quantas unidades de espaço (no eixo X ou no eixo Y) poderá se deslocar.
- Ao instanciar um Robô é necessário informar valores para todas as características citadas.
- O número máximo de movimentos permitidos para qualquer instância de Robô será 30.
- Se for informado um valor fora dos limites permitidos, então o robô deverá ter como limite de movimentos o valor 10.

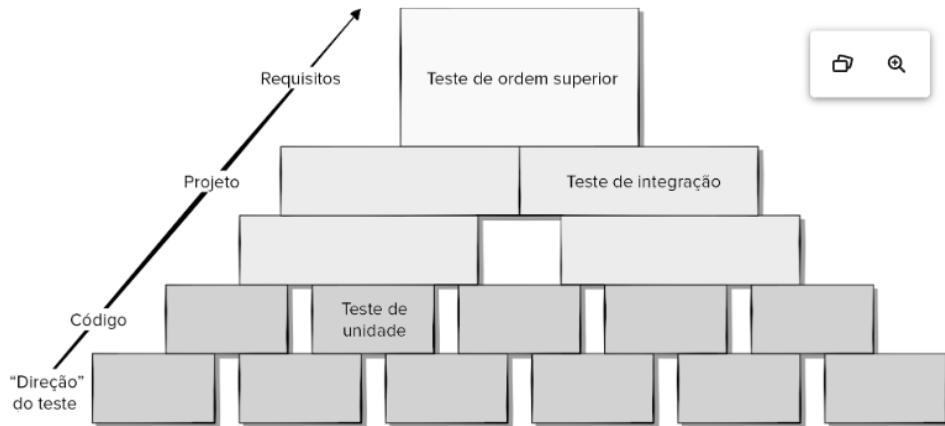
## Exercício: Robô de exploração II

- Para a frente do robô deve-se assumir como 0 graus caso seja informado um valor inválido.
- Para as coordenadas deve-se assumir como (0,0), caso sejam fornecidos valores inválidos para as coordenadas X ou Y.
- O robô deverá ainda prover métodos para:
  - Retornar sua coordenada atual na forma de uma cadeia de caracteres combinando os valores de X e Y, dentro de parênteses e separados por vírgula. Exemplo: "(0,0)";
  - Retornar para onde está apontando sua frente (em graus);
  - Girar o robô em seu próprio eixo, para esquerda ou para a direita. Por exemplo, se o robô estiver com a frente apontada para o 0 graus e for solicitado que gire para esquerda, então o robô ficará apontado para -45 graus. Esse método consome uma unidade da bateria. O método deverá retornar qual é a nova frente do robô após o giro.

## Exercício: Robô de exploração III

- Deslocar  $z$  unidades para onde está apontando sua frente. Por exemplo, se o robô foi instanciado nas coordenadas  $x = 0$ ,  $y = 10$ , frente = 0 graus, então quando este método for invocado, sendo  $z = 20$ , a nova coordenada do robô será:  $x = 0$ ,  $y = 30$ , frente = 0 graus. Antes de iniciar o deslocamento o robô precisa verificar se tem bateria suficiente para isso. Se tiver, então realiza o movimento e o método retorna true, se não tiver, então o robô fica onde está e o método retorna false;
- Retornar o total de movimentos que o robô ainda será capaz de realizar antes que sua bateria acabe.

# Passos do Teste de Software



Fonte: (PRESSMAN; MAXIM, 2021)

## Quando se termina de testar?



# Critérios para Concluir os Testes

- **Errado:** Você nunca termina os testes; o ônus simplesmente muda do engenheiro de software para o usuário final.
- **Errado:** Você termina os testes quando o tempo ou o dinheiro acabam.
- A abordagem de **garantia de qualidade estatística** sugere a execução de testes baseados em uma amostra estatística de todas as possíveis execuções do programa pelos usuários-alvo.
- Coletando métricas durante os testes e utilizando modelos estatísticos existentes é possível desenvolver diretrizes significativas para responder à pergunta: “Quando terminamos os testes?”

# Planejamento de Testes

- Especifique requisitos de produto de forma *quantificável* antes do início dos testes.
- Declare objetivos de teste explicitamente.
- Entenda os usuários do software e desenvolva um perfil para cada categoria.
- Enfatize “testes de ciclo rápido” no planejamento.
- Construa software robusto projetado para se testar automaticamente.
- Use revisões técnicas eficazes como filtro antes dos testes.
- Avalie a estratégia de teste e os casos de teste em revisões técnicas.

# Registro de Testes

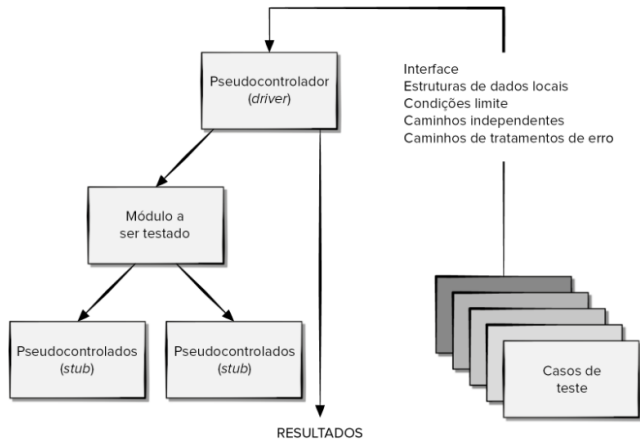
- Casos de teste podem ser registrados em uma planilha (ex.: Google Docs).
- Devem incluir:
  - Uma breve descrição do caso de teste.
  - Um link para o requisito sendo testado.
  - A saída esperada ou critérios de sucesso.
  - Indicação de aprovação ou falha do teste.
  - Datas em que o teste foi executado.
  - Um espaço para comentários sobre falhas (ajuda na depuração).



# O Papel da Infraestrutura de Teste

- Componentes não são programas independentes; algum tipo de infraestrutura é necessária para criar um ambiente de teste.
- Essa infraestrutura inclui *drivers* e/ou *stubs* de software.
- Um **driver** é um “programa principal” que aceita dados de teste, os passa ao componente testado e imprime os resultados.
- Um **stub** substitui módulos chamados pelo componente testado.
  - Usa a interface do módulo.
  - Pode realizar manipulação mínima de dados.
  - Imprime verificações de entrada e retorna controle ao componente testado.

# Ambiente de Teste de Unidade



Fonte: (PRESSMAN; MAXIM, 2021)

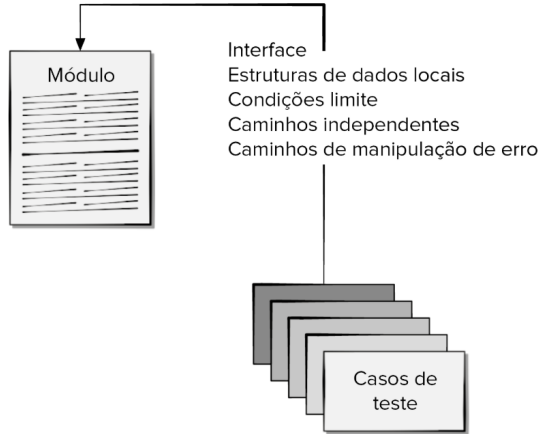
# Testes Eficientes em Termos de Custo

- Testes exaustivos exigiriam que todas as combinações e sequências de entrada fossem processadas.
- O custo-benefício de testes exaustivos geralmente não compensa, pois testar tudo não prova que um componente foi corretamente implementado.
- Os testadores devem ser estratégicos, focando em:
  - Módulos críticos para o sucesso do projeto.
  - Módulos propensos a erros, identificados com base na experiência ou análise.

# Projeto de Casos de Teste

- Desenvolva casos de teste antes de escrever o código do componente.
- Os casos de teste devem abranger:
  - Interface do módulo: verificar entrada e saída de dados corretamente.
  - Estruturas de dados locais: garantir integridade dos dados armazenados.
  - Caminhos independentes: assegurar que todas as instruções sejam executadas ao menos uma vez.
  - Condições de fronteira: testar limites e restrições do processamento.
  - Caminhos de tratamento de erro: garantir respostas adequadas a falhas esperadas.

# Testes de Módulos



Fonte: (PRESSMAN; MAXIM, 2021)

# Tratamento de Erros

- Um bom projeto antecipa condições de erro e estabelece caminhos para seu tratamento.
- Tipos de erro que devem ser testados incluem:
  - Mensagens de erro incompreensíveis.
  - Mensagens de erro que não correspondem ao problema encontrado.
  - Condições de erro que acionam uma intervenção do sistema antes do tratamento adequado.
  - Processamento incorreto de exceções.
  - Mensagens de erro que não fornecem informações suficientes para depuração.

# Rastreabilidade

- Para garantir que o processo de teste seja auditável, cada caso de teste deve estar vinculado a requisitos funcionais ou não funcionais específicos.
- Muitos problemas no processo de teste surgem da falta de rastreabilidade:
  - Caminhos de rastreabilidade ausentes.
  - Dados de teste inconsistentes.
  - Cobertura de teste incompleta.
- Testes de regressão garantem que componentes afetados por mudanças continuem funcionando conforme esperado.

# Teste de Caixa Branca

Usando métodos de teste de caixa branca, você pode derivar casos de teste que:

- 1 Garantam que todos os caminhos independentes dentro de um módulo sejam exercitados ao menos uma vez.
- 2 Exercitem todas as decisões lógicas em seus lados verdadeiro e falso.
- 3 Executem todos os laços em seus limites e dentro de seus valores operacionais.
- 4 Exercitem as estruturas de dados internas para garantir sua validade.



# Teste de Caminhos Básicos

Determine o número de caminhos independentes no programa calculando a Complexidade Ciclomática:

- 1 O número de regiões do grafo de fluxo corresponde à complexidade ciclomática.
- 2 A complexidade ciclomática  $V(G)$  para um grafo de fluxo  $G$  é definida como:

$$V(G) = E - N + 2$$

Onde:

- $E$  é o número de arestas do grafo de fluxo.
- $N$  é o número de nós.

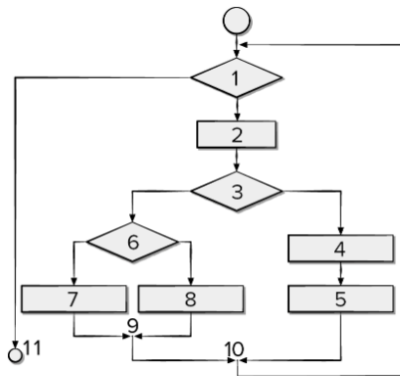
- 3 A complexidade ciclomática  $V(G)$  para um grafo de fluxo  $G$  também é definida como:

$$V(G) = P + 1$$

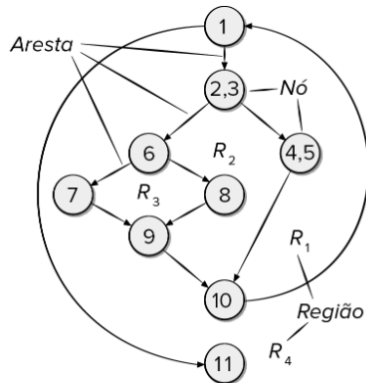
Onde:

- $P$  é o número de nós de decisão (predicados) contidos no grafo de fluxo  $G$ .

# Fluxograma e Grafo de Fluxo



(a)



(b)

Fonte: (PRESSMAN; MAXIM, 2021)

# Teste de Caminhos Básicos

## A Complexidade Ciclomática do grafo de fluxo é 4:

- 1 O grafo de fluxo possui quatro regiões.
- 2  $V(G) = 11 \text{ arestas} - 9 \text{ nós} + 2 = 4$ .
- 3  $V(G) = 3 \text{ nós de decisão} + 1 = 4$ .

**Um caminho independente** é qualquer caminho no programa que introduza ao menos um novo conjunto de instruções de processamento ou uma nova condição.

*(São necessários 4 caminhos independentes para testar.)*

- Caminho 1: 1-11
- Caminho 2: 1-2-3-4-5-10-1-11
- Caminho 3: 1-2-3-6-8-9-10-1-11
- Caminho 4: 1-2-3-6-7-9-10-1-11

# Teste de Caminhos Básicos

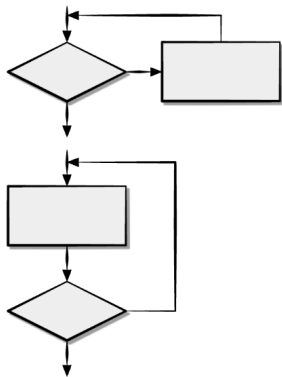
## Projetando Casos de Teste

- Usando o design ou código como base, desenhe um grafo de fluxo correspondente.
- Determine a complexidade ciclomática do grafo de fluxo resultante.
- Identifique um conjunto base de caminhos linearmente independentes.
- Prepare casos de teste que forcem a execução de cada caminho no conjunto base.

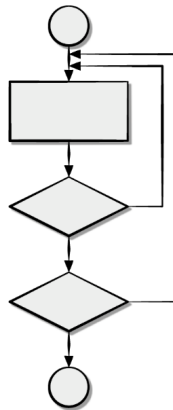
# Teste de Estruturas de Controle

- **Teste de Condição:** Método de design de casos de teste que exercita as condições lógicas contidas em um módulo de programa.
- **Teste de Fluxo de Dados:** Seleciona caminhos de teste de um programa com base nas localizações de definições e usos de variáveis no programa.
- **Teste de Laço:** Técnica de teste de caixa branca que se concentra exclusivamente na validade de construções de laços.

# Classes de Laços



Ciclos simples



Ciclos aninhados

Fonte: (PRESSMAN; MAXIM, 2021)

# Teste de Laços

## Casos de teste para laços simples:

- 1 Ignorar o laço completamente.
- 2 Passar pelo laço apenas uma vez.
- 3 Passar pelo laço duas vezes.
- 4  $m$  iterações no laço, onde  $m < n$ .
- 5  $n - 1$ ,  $n$ , e  $n + 1$  iterações no laço.

# Teste de Laços

## Casos de teste para laços aninhados:

- 1 Comece pelo laço mais interno. Configure os outros laços para os valores mínimos.
- 2 Conduza testes de laços simples no laço mais interno, mantendo os laços externos com valores mínimos de iteração (por exemplo, contador de laços).
- 3 Adicione outros testes para valores fora do intervalo ou excluídos.
- 4 Trabalhe de dentro para fora, conduzindo testes para o próximo laço, enquanto mantém os outros laços externos com valores mínimos e os laços aninhados em valores "típicos".
- 5 Continue até que todos os laços tenham sido testados.



# Referências I



PIRSIG, Robert M. **Zen e a arte da manutenção de motocicletas**: Uma investigação sobre valores. 1. ed.: Presença, 1974.



PRESSMAN, Roger S.; MAXIM, Bruce R. **Engenharia de Software: uma abordagem profissional**. Porto Alegre: Bookman, 2021.  
<https://app.minhabiblioteca.com.br/books/9786558040118/>.